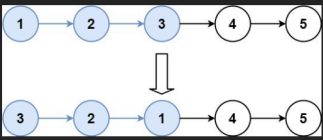


1

leetcode.com/problems/reverse-nodes-in-k-group/

Description | Editorial | Solutions | Submissions



Input: head = [1,2,3,4,5], k = 3
Output: [3,2,1,4,5]

Constraints:

- The number of nodes in the list is n .
- $1 \leq k \leq n \leq 5000$
- $0 \leq \text{Node.val} \leq 1000$

Follow-up: Can you solve the problem in $O(1)$ extra memory space?

Seen this question in a real interview before? 1/5

Accepted 1,477,355 65.4%

Topics: Linked List, Two Pointers

Companies: Amazon, Microsoft, Google, Facebook, Apple, LinkedIn, Oracle, Uber, Lyft, Airbnb, Twitter, Instagram, Pinterest, Slack, Dropbox, Microsoft, Google, Facebook, Apple, LinkedIn, Oracle, Uber, Lyft, Airbnb, Twitter, Instagram, Pinterest, Slack, Dropbox

Discussion (332)

15.5K 332 142 Online

Code

```
C++
1 int length = 0;
2 while (current != nullptr) {
3     length++;
4     current = current->next;
5 }
6
7 while (length >= k) {
8     current = prev->next;
9     ListNode* next = current->next;
10
11     for (int i = 1; i < k; i++) {
12         current->next = next->next;
13         next->next = prev->next;
14         prev->next = next;
15         next = current->next;
16     }
17     prev = current;
18     length -= k;
19 }
20 return dummy->next;
21 }
```

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

head = [1,2,3,4,5]

k = 2

Output

[2,1,4,3,5]

Expected

[2,1,4,3,5]

2

leetcode.com/problems/remove-duplicates-from-sorted-array/

Description | Editorial | Solutions | Submissions

Constraints:

- $1 \leq \text{nums.length} \leq 3 \cdot 10^4$
- $-100 \leq \text{nums}[i] \leq 100$
- nums** is sorted in non-decreasing order.

Seen this question in a real interview before? 1/5

Accepted 7,544,155 62.3%

Topics: Array, Two Pointers

Companies: Amazon, Microsoft, Google, Facebook, Apple, LinkedIn, Oracle, Uber, Lyft, Airbnb, Twitter, Instagram, Pinterest, Slack, Dropbox

Discussion (1.1K)

18.8K 1.1K 495 Online

Code

```
C++
1 class Solution {
2 public:
3     int removeDuplicates(vector<int>& nums) {
4         if (nums.empty()) return 0;
5
6         int k = 1;
7
8         for (int i = 1; i < nums.size(); i++) {
9             if (nums[i] != nums[i-1]) {
10                 nums[k] = nums[i];
11                 k++;
12             }
13         }
14         return k;
15     }
16 };
17
```

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

nums = [1,1,2]

Output

[1,2]

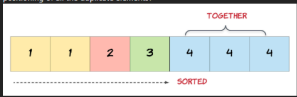
Expected

[1,2]

Contribute a testcase

Hint 1

In this problem, the key point to focus on is the input array being sorted. As far as duplicate elements are concerned, what is their positioning in the array when the given array is sorted? Look at the image below for the answer. If we know the position of one of the elements, do we also know the positioning of all the duplicate elements?



Hint 2

We need to modify the array in-place and the size of the final array would potentially be smaller than the size of the input array. So, we ought to use a two-pointer approach here. One, that would keep track of the current element in the original array and another one for just the unique elements.

Hint 3

Essentially, once an element is encountered, you simply need to **bypass** its duplicates and move on to the next unique element.

3

27. Remove Element

Given an integer array `nums` and an integer `val`, remove all occurrences of `val` in `nums` in-place. The order of the elements may be changed. Then return the number of elements in `nums` which are not equal to `val`.

Consider the number of elements in `nums` which are not equal to `val` be `k`, to get accepted, you need to do the following things:

- Change the array `nums` such that the first `k` elements of `nums` contain the elements which are not equal to `val`. The remaining elements of `nums` are not important as well as the size of `nums`.
- Return `k`.

Custom Judge:

The judge will test your solution with the following code:

```
int[] nums = {...}; // Input array
int val = ...; // Value to remove
int[] expectedNums = {...}; // The expected answer with correct length.
// It is sorted with no values equaling val.

int k = removeElement(nums, val); // Calls your implementation

assert k == expectedNums.length;
sort(nums, 0, k); // Sort the first k elements of nums
for (int i = 0; i < actualLength; i++) {
    assert nums[i] == expectedNums[i];
}
```

If all assertions pass, then your solution will be accepted.

Example 1:

Input: `nums = [3,2,2,3], val = 3`
Output: `2, nums = [2,2,_,_]`
Explanation: Your function should return `k = 2`, with the first two elements of `nums` being 2. It does not matter what you leave beyond the returned `k` (hence they are underscores).

Example 2:

Input: `nums = [0,1,2,2,3,0,4,2], val = 2`
Output: `5, nums = [0,1,4,0,3,_,_,_]`
Explanation: Your function should return `k = 5`, with the first five elements of `nums` containing 0, 1, 4, 0, 3, and 4. Note that the five elements can be returned in any order.

```
class Solution {
public:
    int removeElement(vector<int>& nums, int val) {
        int k = 0;

        for (int i = 0; i < nums.size(); i++) {
            if (nums[i] != val) {
                nums[k] = nums[i];
                k++;
            }
        }

        return k;
    }
};
```

Accepted

Runtime: 0 ms

Case 1 Case 2

Input

`nums = [3,2,2,3]`

`val = 3`

Output

`[2,2]`

Expected

`[2,2]`

4

28. Find the Index of the First Occurrence in a String

Given two strings `needle` and `haystack`, return the index of the first occurrence of `needle` in `haystack`, or `-1` if `needle` is not part of `haystack`.

Example 1:

Input: `haystack = "sadbutsad", needle = "sad"`
Output: `0`
Explanation: "sad" occurs at index 0 and 6. The first occurrence is at index 0, so we return 0.

Example 2:

Input: `haystack = "leetcode", needle = "leeto"`
Output: `-1`
Explanation: "leeto" did not occur in "leetcode", so we return -1.

Constraints:

- `1 <= haystack.length, needle.length <= 104`
- `haystack` and `needle` consist of only lowercase English characters.

Seen this question in a real interview before? 1/5

Accepted 3,924,865 6.5M Acceptance Rate 46.3%

Topics

Companies

Similar Questions

Discussion (512)

```
class Solution {
public:
    int strStr(string haystack, string needle) {
        if (needle.empty()) return 0;

        int n = haystack.length();
        int m = needle.length();

        for (int i = 0; i <= n - m; i++) {
            int j = 0;
            while (j < m && haystack[i + j] == needle[j]) {
                j++;
            }
            if (j == m) {
                return i;
            }
        }

        return -1;
    }
};
```

Accepted

Runtime: 0 ms

Case 1 Case 2

Input

`haystack = "sadbutsad"`

`needle = "sad"`

Output

`0`

Expected

`0`

5

29. Divide Two Integers

Medium

Given two integers *dividend* and *divisor*, divide two integers **without** using multiplication, division, and mod operator.

The integer division should truncate toward zero, which means losing its fractional part. For example, $8/345$ would be truncated to 0, and -2.7335 would be truncated to -2 .

Return the **quotient** after dividing *dividend* by *divisor*.

Note: Assume we are dealing with an environment that could only store integers within the 32-bit signed integer range: $[-2^{31}, 2^{31} - 1]$. For this problem, if the quotient is **strictly greater than** $2^{31} - 1$, then return $2^{31} - 1$, and if the quotient is **strictly less than** -2^{31} , then return -2^{31} .

Example 1:

Input: dividend = 10, divisor = 3
Output: 3
Explanation: $10/3 = 3.3333\ldots$, which is truncated to 3.

Example 2:

Input: dividend = 7, divisor = -3
Output: -2
Explanation: $7/-3 = -2.3333\ldots$, which is truncated to -2.

Constraints:

- $-2^{31} \leq \text{dividend}, \text{divisor} \leq 2^{31} - 1$
- $\text{divisor} \neq 0$

Seen this question in a real interview before? 1/5

Accepted 1,121,356 5.8M Acceptance Rate 19.4%

Topics: Companies

6.1K 450 97 Online

```

C++
17 while (absDividend >= (temp << 1)) {
18     temp <<= 1;
19     multiple <<= 1;
20 }
21 absDividend -= temp;
22 quotient += multiple;
23 }
24 if (negative) {
25     quotient = -quotient;
26 }
27 if (quotient > INT_MAX) {
28     return INT_MAX;
29 }
30 if (quotient < INT_MIN) {
31     return INT_MIN;
32 }
33 return quotient;
34 }
35 }
36 }
37 }
38 }
39 }
40 }

```

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

dividend = 10

divisor = 3

Output

3

Expected

3

6

30. Substring with Concatenation of All Words

Hard

You are given a string *s* and an array of strings *words*. All the strings of *words* are of the same length.

A **concatenated string** is a string that exactly contains all the strings of any permutation of *words* concatenated.

- For example, if *words* = ["ab", "cd", "ef"], then "abcdef", "abefcd", "cdabef", "cdefab", "efabcd", and "efcdab" are all concatenated strings. "acdbef" is not a concatenated string because it is not the concatenation of any permutation of *words*.

Return an array of the starting indices of all the concatenated substrings in *s*. You can return the answer in **any order**.

Example 1:

Input: s = "barfoothefoobarman", words = ["foo", "bar"]
Output: [0, 9]
Explanation:
The substring starting at 0 is "barfoo". It is the concatenation of ["bar", "foo"] which is a permutation of *words*.
The substring starting at 9 is "thefoobar". It is the concatenation of ["foo", "bar"] which is a permutation of *words*.

Example 2:

Input: s = "wordgoodgoodgoodbestword", words = ["word", "good", "best", "word"]
Output: []
Explanation:
There is no concatenated substring.

Example 3:

Input: s = "barfoothefoobarman", words = ["bar", "foo", "the"]
Output: [6, 9, 12]
Explanation:
The substring starting at 6 is "foothebar". It is the concatenation of ["foo", "bar", "the"].
The substring starting at 9 is "barthefoo". It is the concatenation of ["bar", "the", "foo"].
The substring starting at 12 is "thefoobar". It is the concatenation of ["the", "foo", "bar"].

```

C++
30 while (seen[word] > wordFreq[word]) {
31     string leftWord = s.substr(left, wordLen);
32     seen[leftWord]--;
33     count--;
34     left += wordLen;
35 }
36 if (count == wordCount) {
37     result.push_back(left);
38 }
39 else {
40     seen.clear();
41     count = 0;
42     left = right;
43 }
44 }
45 return result;
46 }
47 }
48 }
49 }
50 }
51 }
52 }
53 }

```

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

s = "barfoothefoobarman"

words = ["foo", "bar"]

Output

[0, 9]

Expected

[0, 9]

7

8

1